

Билет 19. Lock-free очередь Майкла-Скотта.

ABA problem

0. Формулировка билета

Lock-free очередь — очередь Майкла-Скотта — и её реализация. Проблема ABA, решение этой проблемы.

Главная идея:

Очередь Майкла-Скотта — классическая lock-free очередь на односвязном списке с двумя атомарными указателями: `head` и `tail`.

Обычная очередь:

```
push/enqueue — добавить в конец  
pop/dequeue  — снять из начала
```

В lock-free очереди нельзя просто защитить всю структуру mutex-ом.

Поэтому используются:

```
atomic head  
atomic tail  
CAS  
dummy node  
helping
```

1. Почему очередь сложнее стека

В стеке Трейбера была одна главная точка конкуренции:

```
atomic head
```

`push` и `pop` оба меняли `head`.

В очереди логически две стороны:

```
enqueue — работает с tail  
dequeue  — работает с head
```

Это хорошо: добавления и удаления могут меньше мешать друг другу.

Но структура сложнее, потому что нужно поддерживать связанный список:

```
head → node1 → node2 → node3 ← tail
```

При добавлении нового узла в конец надо сделать две вещи:

1. Привязать новый узел к старому tail через tail->next.
2. Передвинуть tail на новый узел.

Проблема:

Эти две операции нельзя сделать одной обычной атомарной операцией, если у нас есть только CAS над одним указателем.

Поэтому алгоритм устроен так, что очередь остаётся корректной даже если поток успел выполнить только часть операции, а другой поток потом ему “помог”.

2. Dummy node

Очередь Майкла–Скотта обычно начинается с фиктивного узла — **dummy node**.

Пустая очередь выглядит так:

```
head
↓
dummy
↑
tail
```

То есть:

```
head == tail
head->next == nullptr
```

Зачем нужен dummy node?

Чтобы упростить случаи пустой очереди и удаления.

В такой очереди `head` указывает не на первый настоящий элемент, а на dummy node.

Первый настоящий элемент — это:

```
head->next
```

Например, очередь с элементами **A**, **B**, **C**:

```
head → dummy → A → B → C
                        ↑
                    tail
```

При **dequeue** мы обычно берём значение из **head->next**, а затем двигаем **head** вперёд.

После удаления **A**:

```
head → A → B → C
        ↑
    tail
```

Старый dummy можно удалить позже, а узел **A** становится новым dummy.

3. Структура узла

Узел очереди:

```
template <typename T>
struct Node {
    T value;
    std::atomic<Node*> next;

    Node(T v)
        : value(std::move(v)), next(nullptr)
    {}
};
```

У очереди есть:

```
std::atomic<Node*> head_;
std::atomic<Node*> tail_;
```

Изначально:

```
Node* dummy = new Node(...);
head_.store(dummy);
tail_.store(dummy);
```

В реальном коде для dummy node часто отдельно обрабатывают отсутствие значения, например через `std::optional<T>`.

4. Основная идея enqueue

Хотим добавить новый узел в конец.

Было:

```
head → dummy → A → B
                        ↑
                      tail
```

Добавляем `X`.

Нужно получить:

```
head → dummy → A → B → X
                        ↑
                      tail
```

Но это делается в две фазы:

- ```
1. CAS(tail->next, nullptr, new_node)
2. CAS(tail, old_tail, new_node)
```

Первая операция действительно добавляет узел в список.

Вторая операция только продвигает хвостовой указатель.

Главная мысль:

Логически enqueue произошёл, когда мы успешно привязали новый узел через `old_tail->next`.

То есть linearization point для enqueue — успешный CAS по `tail->next`.

---

## 5. Почему tail может отставать

Представим, что поток сделал:

```
CAS(old_tail->next, nullptr, new_node) – успешно
```

Список уже стал:

```
head → dummy → A → B → X
 ↑
 tail
```

`X` уже в очереди, но `tail` ещё указывает на `B`.

Поток хотел потом сделать:

```
CAS(tail, B, X)
```

но его вытеснил scheduler.

Очередь находится в промежуточном, но корректном состоянии:

```
tail отстаёт от настоящего конца списка
```

Другие потоки могут это увидеть.

Идея алгоритма:

Если поток видит, что `tail->next != nullptr`, значит tail отстал, и можно помочь продвинуть tail вперёд.

Это называется **helping**.

---

## 6. Enqueue в очереди Майкла-Скотта

Схема:

```
enqueue(x):
 new_node = new Node(x)

 loop:
 tail = tail_.load()
 next = tail->next.load()
```

```

если tail всё ещё актуален:
 если next == nullptr:
 tail действительно последний
 пробуем CAS(tail->next, nullptr, new_node)

 если успешно:
 пробуем CAS(tail_, tail, new_node)
 return

 иначе:
 tail отстал
 помогаем: CAS(tail_, tail, next)

```

Упрощённый код:

```

void enqueue(T value) {
 Node* new_node = new Node(std::move(value));

 while (true) {
 Node* tail = tail_.load(std::memory_order_acquire);
 Node* next = tail->next.load(std::memory_order_acquire);

 if (tail == tail_.load(std::memory_order_acquire)) {
 if (next == nullptr) {
 if (tail->next.compare_exchange_weak(
 next,
 new_node,
 std::memory_order_release,
 std::memory_order_relaxed
)) {
 tail_.compare_exchange_weak(
 tail,
 new_node,
 std::memory_order_release,
 std::memory_order_relaxed
);

 return;
 }
 } else {
 tail_.compare_exchange_weak(
 tail,
 next,
 std::memory_order_release,
 std::memory_order_relaxed
);
 }
 }
 }
}

```

```
}
}
```

Ключевые идеи:

1. Новый узел реально вставляется через CAS на `tail->next`.
2. Обновление `tail` – вспомогательная оптимизация.
3. Если `tail` отстал, другой поток может помочь его продвинуть.

## 7. Основная идея dequeue

В очереди с dummy node `head` указывает на фиктивный или уже использованный узел.

Первый настоящий элемент:

```
first = head->next
```

Например:

```
head → dummy → A → B
 ↑
 tail
```

`dequeue` должен удалить `A`.

После удаления:

```
head → A → B
 ↑
 tail
```

Узел `A` становится новым dummy.

Логически мы сняли значение `A`.

## 8. Dequeue: основные случаи

В цикле читаем:

```
head = head_.load()
tail = tail_.load()
next = head->next.load()
```

Дальше возможны случаи.

### Случай 1. Очередь пустая

```
head == tail
next == nullptr
```

Это значит:

очередь пуста

Возвращаем пустой результат.

---

### Случай 2. Tail отстал

```
head == tail
next != nullptr
```

Это значит:

очередь не пуста,  
но tail всё ещё указывает на старый dummy

Тогда поток помогает:

```
CAS(tail, tail, next)
```

То есть продвигает tail.

---

### Случай 3. Есть настоящий элемент

```
head != tail
next != nullptr
```

Тогда первый настоящий элемент — `next`.

Пытаемся сдвинуть head:

```
CAS(head, old_head, next)
```

Если CAS успешен:

```
мы логически удалили элемент next
возвращаем next->value
старый head можно будет освободить позже
```

Linearization point для успешного dequeue — успешный CAS по `head`.

## 9. Dequeue в очереди Майкла-Скотта

Упрощённый код:

```
std::optional<T> dequeue() {
 while (true) {
 Node* head = head_.load(std::memory_order_acquire);
 Node* tail = tail_.load(std::memory_order_acquire);
 Node* next = head->next.load(std::memory_order_acquire);

 if (head == head_.load(std::memory_order_acquire)) {
 if (head == tail) {
 if (next == nullptr) {
 return std::nullopt;
 }

 tail_.compare_exchange_weak(
 tail,
 next,
 std::memory_order_release,
 std::memory_order_relaxed
);
 } else {
 T result = next->value;

 if (head_.compare_exchange_weak(
 head,
 next,
 std::memory_order_release,
 std::memory_order_relaxed
)) {
 // Старый head удалять сразу опасно.
 // Нужна memory reclamation.
 delete head;
 }
 }
 }
 }
}
```

```

 return result;
 }
}
}
}

```

Важно:

`delete head` здесь так же опасен, как и в Treiber stack.  
В настоящем lock-free коде нужна схема memory reclamation.

## 10. Почему dequeue берёт value до CAS

В классическом варианте часто сначала читают:

```
T result = next->value;
```

а потом делают CAS `head → next`.

Почему?

Потому что после успешного CAS другой поток может освободить старые узлы или изменить ситуацию вокруг.

Но это место тонкое: в настоящей реализации нужно думать о lifetime узла `next`.

Для экзамена можно сказать проще:

В `dequeue` мы читаем первый настоящий узел `next = head->next`, а операция считается совершённой в момент успешного CAS, который передвигает `head` на `next`.

## 11. Почему очередь Майкла–Скотта lock-free

Алгоритм состоит из CAS-циклов.

Если мой CAS не прошёл, значит кто-то другой изменил:

```
tail->next
tail
head
```

То есть кто-то сделал прогресс.

Кроме того, есть helping:

если tail отстал, другой поток может продвинуть его

Это важный аргумент для lock-free.

Формула:

CAS failed или увидели tail lag:  
значит другой поток уже продвинул структуру  
или мы можем помочь завершить его операцию

Поэтому очередь Майкла-Скотта lock-free:

система в целом делает прогресс

Но она не обязательно wait-free:

конкретный поток может долго проигрывать CAS

---

## 12. Зачем нужен helping

Helping нужен, потому что операция enqueue логически состоит из двух шагов:

1. привязать новый узел через tail->next
2. продвинуть tail

Поток может успеть сделать шаг 1 и остановиться.

Без helping другие потоки могли бы постоянно видеть "странное" состояние.

С helping всё нормально:

tail->next != nullptr  
значит tail отстал  
любой поток может сделать CAS(tail, tail, tail->next)

Так структура не зависит от того, завершит ли конкретный поток свою операцию.

Это помогает обеспечить lock-free progress.

---

## 13. Linearization points

Важно уметь назвать linearization point.

### Enqueue

Linearization point:

успешный CAS, который меняет `old_tail->next` с `nullptr` на `new_node`

Почему не CAS по `tail`?

Потому что после успешного CAS по `tail->next` новый узел уже достигим из `head`.

Даже если `tail` ещё не передвинут, элемент уже находится в очереди.

---

### Dequeue

Linearization point:

успешный CAS, который меняет `head` с `old_head` на `next`

Именно в этот момент первый элемент считается снятым из очереди.

---

### Empty dequeue

Если `dequeue` увидел:

```
head == tail
next == nullptr
```

то очередь пуста.

Linearization point — чтение `head`, `tail`, `next`, которое согласованно показало пустое состояние.

На устном можно не углубляться, но полезно знать.

---

## 14. ABA problem

ABA problem возникает, когда CAS проверяет только значение указателя, но не знает историю изменений.

Общий сценарий:

```
поток 1 видит значение A
потом другие потоки меняют A → B → A
поток 1 снова видит A
CAS думает, что ничего не изменилось
```

Но структура могла полностью измениться.

CAS видит:

```
A == A
```

но не видит:

```
между ними было B
```

---

## 15. ABA в lock-free структурах

В структурах на указателях это особенно опасно.

Например, в стеке:

```
head = A
A->next = B

thread 1:
 old_head = A
 next = B
 pause

thread 2:
 pop A
 pop B
 push A обратно

thread 1:
 CAS(head, A, B) проходит
```

Но **B** уже может не быть в стеке или вообще быть освобождён.

Структура повреждается.

В очереди АВА тоже может возникать с указателями **head**, **tail**, **next**, особенно если узлы освобождаются и переиспользуются.

---

## 16. АВА и memory reclamation связаны

Очень часто АВА возникает из-за переиспользования памяти.

Например:

```
узел А удалили
память А освободили
аллокатор позже выдал тот же адрес новому узлу
указатель снова равен А
```

Для CAS это выглядит как “то же самое значение”.

Но логически это уже другой объект.

Поэтому memory reclamation важна не только против use-after-free, но и для уменьшения риска АВА.

---

## 17. Способы борьбы с АВА

### 1. Tagged pointer

Храним не только указатель, но и версию:

```
(pointer, counter)
```

Каждое изменение увеличивает counter.

Было:

```
(A, 1)
```

Потом:

```
(B, 2)
```

Потом снова адрес `A`, но версия новая:

`(A, 3)`

CAS сравнивает пару:

`(A, 1) != (A, 3)`

Поэтому ABA обнаруживается.

Минус:

нужен CAS по паре `pointer+counter`  
иногда нужен `double-width CAS`

---

## 2. Hazard pointers

Поток публикует:

я сейчас использую указатель `A`

Пока `A` находится в hazard pointer какого-то потока, другие потоки не освобождают этот узел.

Это не всегда полностью решает логическую ABA, но сильно помогает против переиспользования освобождённой памяти.

---

## 3. Epoch-based reclamation

Память не освобождается сразу.

Удалённые узлы кладутся в `retired list`.

Освобождать их можно только когда все потоки вышли из эпохи, в которой они могли видеть старые указатели.

Идея:

не удалять узел, пока кто-то теоретически может держать на него указатель

## 4. Reference counting

У узлов может быть счётчик ссылок.

Пока кто-то работает с узлом, он увеличивает счётчик.

Удалять можно только когда счётчик стал нулём.

Минус:

дополнительные атомарные операции  
сложность  
overhead

---

## 5. Garbage collection / managed memory

В языках с GC память не будет освобождена, пока объект достижим.

Это помогает против use-after-free и переиспользования адресов.

Но логическая ABA всё равно иногда возможна, если объект удалили из структуры и потом вставили обратно.

---

## 18. Memory reclamation в очереди

В `dequeue` после успешного CAS старый dummy node больше не нужен логически.

Наивно хочется сделать:

`delete old_head;`

Но другой поток мог раньше прочитать `old_head` и ещё работать с ним.

Сценарий:

thread A:  
  head = head\_.load()  
  pause  
  
thread B:  
  dequeue()  
  CAS head → next  
  delete old\_head

```
thread A:
 продолжает читать head->next
 но head уже удалён
```

Получаем use-after-free.

Поэтому очередь Майкла-Скотта, как и стек Трайбера, требует безопасной схемы освобождения памяти.

---

## 19. Минимальная схема для устного ответа

Если мало времени, очередь Майкла-Скотта можно объяснить так:

Очередь хранит `atomic head` и `atomic tail` и использует `dummy node`.

`head` указывает на `dummy`, первый настоящий элемент — `head->next`.  
`tail` указывает примерно на последний элемент, но может отставать.

`enqueue` создаёт новый узел и пытается CAS-ом поставить его в `tail->next`.  
Если получилось, элемент уже в очереди. Потом поток пытается продвинуть `tail`.  
Если другой поток видит, что `tail->next != nullptr`, он помогает продвинуть `tail`.

`dequeue` смотрит `head`, `tail` и `head->next`.  
Если `head == tail` и `next == nullptr`, очередь пуста.  
Если `head == tail`, но `next != nullptr`, `tail` отстал, надо помочь продвинуть `tail`.  
Иначе первый элемент — `next`, и мы CAS-ом двигаем `head` на `next`.

Успешный CAS на `tail->next` — `linearization point enqueue`.  
Успешный CAS на `head` — `linearization point dequeue`.

Это занимает меньше минуты и закрывает главное.

---

## 20. Сравнение со стеком Трайбера

Стек Трайбера:

```
один atomic head
push/pop конкурируют за head
алгоритм проще
```

Очередь Майкла-Скотта:

```
atomic head + atomic tail
enqueue работает с tail
dequeue работает с head
есть dummy node
tail может отставать
нужен helping
```

Обе структуры:

```
lock-free
CAS-based
не обязательно wait-free
имеют ABA/memory reclamation проблемы
```

---

## 21. Что сказать коротко

Очередь Майкла-Скотта — классическая lock-free FIFO-очередь на односвязном списке. Она хранит атомарные указатели `head` и `tail` и использует dummy node. `head` указывает на фиктивный узел, а первый настоящий элемент — это `head->next`. `tail` указывает на последний узел, но может временно отставать.

`enqueue` создаёт новый узел и пытается CAS-ом заменить `tail->next` с `nullptr` на этот новый узел. Если CAS успешен, элемент уже логически добавлен в очередь. После этого поток пытается продвинуть `tail` на новый узел. Если другой поток видит, что `tail->next != nullptr`, он понимает, что `tail` отстал, и помогает продвинуть его.

`dequeue` читает `head`, `tail` и `head->next`. Если `head == tail` и `next == nullptr`, очередь пуста. Если `head == tail`, но `next != nullptr`, значит `tail` отстал, и поток помогает его продвинуть. Иначе `next` — первый настоящий элемент, и поток CAS-ом пытается передвинуть `head` на `next`. Успешный CAS по `head` — момент, когда элемент логически удалён.

Очередь Майкла-Скотта lock-free, потому что если CAS одного потока не проходит, значит другой поток изменил структуру; кроме того, потоки могут помогать друг другу продвигать `tail`. Но очередь не wait-free, потому что конкретный поток может долго проигрывать CAS.

ABA problem возникает, когда CAS видит одинаковое значение указателя, хотя между чтением и CAS структура изменилась: было `A`, стало `B`, потом снова `A`. CAS сравнивает только значение и не видит историю. Решения: tagged pointers с версией, hazard pointers, epoch-based reclamation, reference counting, иногда garbage collection. Также нужна memory reclamation, потому что нельзя сразу удалять узлы после `dequeue`: другой поток мог раньше прочитать указатель на удаляемый узел.

---

## 22. Мини-проверка

1. Почему lock-free очередь сложнее lock-free стека?
2. Зачем в очереди Майкла-Скотта нужен dummy node?
3. На что указывают `head` и `tail`?
4. Почему `tail` может отставать?
5. Как устроен `enqueue` в очереди Майкла-Скотта?
6. Как устроен `dequeue`?
7. Что означает ситуация `head == tail` и `next == nullptr`?
8. Что означает ситуация `head == tail`, но `next != nullptr`?
9. Что такое helping?
10. Где linearization point у `enqueue` и `dequeue`?
11. Что такое ABA problem?
12. Какие есть способы борьбы с ABA?